

Explain what goes wrong in this computation in floating point, and rewrite the problematic formula in a new function `pi_estimator2` to avoid the issue. Show a plot of the relative error similar to the one above to demonstrate that you have resolved the problem.

In this computation of floating point, what goes wrong is cancellation, because we subtract two floating point numbers that are nearly equal, like 1 and $\sqrt{1 - \frac{L^2}{4}}$ (intuitively, for an n-gon in a unit circle, the side lengths will tend to 0, so $\sqrt{1 - \frac{L^2}{4}}$ will also tend to 0). This subtraction makes most leading digits cancel, which leaves us with very small values that are directly affected by the limited precision in computers.

We must change the expression to avoid this cancellation. In the next algorithm, an equivalent formula is used without this subtraction, which can approximate π nicely.

The original expression is expanded out to be $\sqrt{2 - \sqrt{4 - L^2}}$ (Still has this cancellation problem by similar reasoning, L will go to 0 and it will lead to cancellation between 2 and $\sqrt{4} = 2$, and to prevent cancellation:

(Multiply the expression by a fancy 1 (algebra trick/diff of squares), $\sqrt{2 - \sqrt{4 - L^2}} * \frac{\sqrt{2 + \sqrt{4 - L^2}}}{\sqrt{2 + \sqrt{4 - L^2}}}$), which will remove the pesky cancellation problem and result in the expression: $\frac{L}{\sqrt{2 + \sqrt{4 - L^2}}}$, which doesn't have cancellation (no near zero terms when subtracting)

2. Conditional probabilities

Suppose $Z \sim N(0, 1)$ is a standard normal random variable. The following computation of $P\{Z < -50.1 | Z < -50\}$ clearly fails. Explain why, and find an alternate expression (using the functions defined in [StatsFuns.jl](#)) that should give reasonable accuracy.

(Julia uses a double precision, 64 bit float format, called Float64)

This above computation is invalid and return NaN because it actually involves o/o, since both $\text{normcdf}(-50.1)$ and $\text{normcdf}(-50)$ are under the the smallest subnormal representation by Float64 (of 2^{-1074}) and when a number is under the smallest subnormal, it is automatically rounded to 0, which causes the o/o.

An alternative expression that would give rise to reasonable accuracy would be the one below because normlogcdf would return nonzero numbers that are in the -1000's (finite) range that won't cancel even if the computer's float value precision is limited, due to the use of logarithmic function, and because 50 and 50.1 are quite close, it reasons that the conditional probability is much more likely not below subnormal too (despite the individual ones being below subnormal).

Mathematically $e^{\log x} = x$, the use of $\log x$ elegantly avoided the trouble incurred by the limited precision of float numbers in a computer (uses subtraction before exponentiation, exponentiates a "nice" number rather than subtracting to basically 0 numbers)

3. Sums and recurrences $\Leftrightarrow$

Suppose $F \in \mathbb{R}^{n \times n}$ satisfies $\|F\| < 1$ for some operator norm. Then

1. Argue that the iteration $x_{k+1} = Fx_k + b$ starting from $x_0 = 0$ computes $x_k = S_k b$ where $S_k = \sum_{j=0}^{k-1} F^j$ is the k th partial sum of the Neumann series for $(I - F)^{-1}$.
2. If $x_{k+1} = Fx_k + y_k$ starting from $x_0 = 0$ where $\|y_k\| \leq \eta$ for all k , then show by induction that $\|x_k\| \leq \eta/(1 - \|F\|)$ for all $k \geq 0$.

Proof of 1.

Base case we know that $x_0 = 0$ so by the first formula, $x_1 = Fx_0 + b$ which is equal to b , and by the second formula, $x_k = S_k b$ and $S_k = \sum_{j=0}^0 F^j = 1$ which is also equal to b , thus these two expressions are equivalent.

Inductive Hypothesis (IH) Assume that the two expressions, are equal, such that $x_k = S_k b = Fx_{k-1} + b$ is true when $n = k$

Inductive Step Based on the IH, prove the two expressions are of the same value when $n = k + 1$.

For the first expression, $x_{k+1} = Fx_k + b$, using IH we can substitute $x_k = S_k b$ so that it becomes $x_{k+1} = F(S_k b) + b$. For the second expression, by expanding $S_{k+1} = \sum_{j=0}^k F^j = F(\sum_{j=0}^{k-1} F^j) + 1$, and by definition $S_k = \sum_{j=0}^{k-1} F^j$, so the second expression also evaluates to $x_{k+1} = F(S_k b) + b$, x_{k+1} in both expressions are again equal. The inductive step thus holds.

With the base case and inductive step proved, the claim is hence proved.

Proof of 2.

Base case we know that $x_0 = 0$, and in expression $\eta/(1 - \|F\|)$: since $\|F\| < 1$ and $\|y_k\| \geq 0$ thus $\eta \geq 0$, thereby $\eta/(1 - \|F\|) \geq 0 = \|x_0\|$, the base case hold true.

Inductive Hypothesis Assume the claim is true when $n = k$: $\|x_k\| \leq \eta/(1 - \|F\|)$

Inductive Step prove $n = k + 1$ the inequality holds: $\|x_{k+1}\| \leq \eta/(1 - \|F\|)$

We know that $\|x_{k+1}\| = \|Fx_k + y_k\|$ by the recurrence equation, and by the norm's triangular inequality, $\|Fx_k + y_k\| \leq \|Fx_k\| + \|y_k\|$, and by the operator norm, we know that it must have consistency, so $\|Fx_k\| + \|y_k\| \leq \|F\|\|x_k\| + \|y_k\|$, and we know that $\|y_k\| \leq \eta$ by definition, and using IH:

$$\|x_{k+1}\| \leq \|F\|\|x_k\| + \|y_k\| \leq \frac{\|F\|\eta}{1 - \|F\|} + \eta = \frac{\|F\|\eta}{1 - \|F\|} + \frac{\eta(1 - \|F\|)}{1 - \|F\|} = \eta/(1 - \|F\|)$$

which proved the claim stated in the inductive step.

This concludes the proof of the claim 2.